INNOTECH'26 · KIET GROUP OF INSTITUTIONS · 30 OCT 2026



ModelMind Notebook

Backend Architecture & Technical Deep Dive

CATEGORY	PLATFORM	LANGUAGE	HARDWARE
AI Solutions for Automation	JupyterLab + FastAPI	Python 3.14 · TypeScript	Apple M4 · 16 GB RAM

🖥️ Local LLM (Ollama) · ☁️ Gemini Cloud Fallback · 🔒 Privacy-First · 📦 Sandbox Validation · 🔬 ML-Specialized Diagnostics · ⚡ Deterministic-First Pipeline · 🔑 OS Keychain Security · 🚀 Next.js Dashboard

1 Project Overview & Architecture

What ModelMind Notebook solves and how the two-layer backend is structured

37+	20+	3-Tier	100%
BACKEND MODULES	API ENDPOINTS	AI PIPELINE	OFFLINE CAPABLE

🎯 Problem We Solve

- ML beginners waste hours copy-pasting errors into ChatGPT
- Every other tool (Colab, Kaggle, Deepnote) sends private code to cloud
- No tool validates an AI fix before showing it to the user
- Generic AI gives generic answers — no ML domain awareness

✅ Our Solution

- **Privacy-first:** AI runs locally via Ollama — zero data leaves by default
- **Auto-capture:** Detects failed cell automatically — no copy-pasting
- **Deterministic-first:** Rules solve common ML errors without any AI
- **Sandbox validation:** Every AI fix tested before shown to user
- **Full ML ecosystem:** Preprocessing, model rec, evaluation, tuning

🏗️ Two-Layer Backend Architecture

Layer 1 — Jupyter Server Extension

Python extension inside JupyterLab. Owns all AI routing, error diagnosis, ML pipeline, fix validation. Every route uses `@tornado.web.authenticated`. Runs at `127.0.0.1:8899`.

Layer 2 — FastAPI Companion Server

Standalone FastAPI v0.3.0 server bridging the Next.js web dashboard with local Jupyter runtime. Handles dataset uploads, package management, notebook session linking. Runs at `127.0.0.1:8000`.

2 The 3-Tier AI Routing Pipeline

Core innovation: deterministic → local AI → cloud AI with privacy gates

> 🔑 **Core Innovation**
>
> Unlike every other notebook (Colab, Kaggle, Databricks), ModelMind does NOT immediately call an AI. It first runs deterministic rule-based analysis. AI is called **only when rules are insufficient**. This saves privacy, saves time, and is more accurate for common ML errors.

1 ❌ **Cell Fails — Auto Error Capture (JupyterLab Frontend Extension)**

Extension automatically captures the failing cell — no user action needed. Payload bounded server-side before any processing.

`cell_id (256 chars)` `code (20,000 chars)` `error_type (200 chars)` `traceback (12,000 chars)` `MAX 40,000 bytes total`

2 🔍 **Deterministic Diagnosis — diagnostics.py (Zero AI Called)**

Rule-based pattern matching runs first. Handles 5 major ML error classes. Returns `is_sufficient: True` (instant fix, no AI) or `False` (falls through to AI).

`SyntaxError` `NameError → scope analysis` `KeyError → column hints` `NumPy Shape Mismatch` `sklearn Feature Count Mismatch`

3 🖥️ **Local Ollama AI — qwen2.5-coder:7b — ollama.py**

Contacts local Ollama at `127.0.0.1:11434`. Runs 100% on your machine — no internet. JSON schema-validated response. Bounded timeouts and response sizes.

`Explain timeout: 60s` `Fix timeout: 120s` `Max resp: 32KB` `lru_cache manifest` `Structured JSON schema`

4 📦 **Sandbox Validation — fix_validation.py (Applied to EVERY Fix)**

Before any fix is shown, it runs in a disposable sandbox. Fail → rejected. Pass → diff shown → user must click Apply. Original cell preserved. No auto-application ever.

`ast.parse() syntax check` `SAFE_BUILTINS restricted exec` `difflib diff generation` `is_complete_cell_fix() check`

5 ☁️ **Gemini Cloud Fallback — gemini.py + cloud_policy.py (3 Gates)**

Only if Ollama fails AND all 3 gates pass. Cascade: `gemini-3.5-flash-lite` → `gemini-3.5-flash` → `gemini-flash-latest`. certifi TLS. Browser NEVER receives API key.

`Gate 1: ML_PLATFORM_CLOUD_FALLBACK=enabled` `Gate 2: Server API key configured` `Gate 3: User opts-in per session`

3 Complete API Endpoints — Jupyter Server Extension

All routes authenticated with @tornado.web.authenticated — 403 on any unauthenticated request

🔐 Authentication Model

Every route uses `@tornado.web.authenticated`. All requests require a valid Jupyter Server token. Browser cannot call any AI API directly. 403 returned on unauthenticated attempts.

Method	Endpoint	Handler	What It Does
GET	/api/health	HealthHandler	Platform status, Ollama status, cloud policy state
POST	/api/error/analyze	AnalyzeErrorHandler	Deterministic error analysis — no AI called
POST	/api/ai/explain	ExplainHandler	Ollama/Gemini explanation for complex errors
POST	/api/ai/fix	FixHandler	AI fix → sandbox validated → diff shown to user
POST	/api/ai/code-help	CodeHelpHandler	General code Q&A with local Ollama
POST	/api/ai/validate-fix	ValidateFixHandler	Sandbox-validate a candidate fix code
GET	/api/local-ai/status	LocalAIStatusHandler	Ollama version + model presence check
POST	/api/local-ai/install	InstallHandler	User-approved Ollama install (allow_download flag required)
POST	/api/local-ai/model/ensure	EnsureModelHandler	Download model with explicit approval only
GET	/api/gemini-key	GeminiAPIKeyHandler	Check if Gemini key configured and its source
POST	/api/gemini-key	GeminiAPIKeyHandler	Save API key to OS Keychain — never to browser
DEL	/api/gemini-key	GeminiAPIKeyHandler	Remove API key from OS credential store
POST	/api/dataset/analyze	DatasetHandler	Dataset profiling: shape, dtypes, missing%, distributions
POST	/api/dataset/target	TargetHandler	Target column analysis — classification vs regression
POST	/api/dataset/feature	FeatureHandler	Individual feature analysis with statistical summary
POST	/api/preprocessing/analyze	PreprocessingAdvisorHandler	Smart per-column preprocessing recommendations
POST	/api/preprocessing/preview	PreprocessingPreviewHandler	Preview preprocessing result before applying
POST	/api/preprocessing/apply	PreprocessingApplyHandler	Apply transformations and save cleaned dataset
POST	/api/preprocessing/undo	PreprocessingUndoHandler	Revert last preprocessing step (snapshot-based)
POST	/api/ml/mistakes	MistakeDetectorHandler	Static AST-based ML methodology mistake detection
POST	/api/models/recommend	ModelRecommenderHandler	Rank-ordered ML model recommendations with ready-to-run code

4 Core Backend Python Modules

37+ modules — single responsibility, thoroughly tested

AI & INTELLIGENCE LAYER

diagnostics.py

Rule-based ML error classifier. SyntaxError, NameError, KeyError, NumPy shape, sklearn feature count. Returns `is_sufficient` flag to gate all AI usage.

ollama.py

Local LLM connector. Health checks via /api/version + /api/tags. Structured JSON schema output. lru_cache manifest. Bounded timeouts + response sizes.

gemini.py

Cloud fallback AI. 3-gate policy. Model cascade (flash-lite → flash → latest). certifi TLS. Exponential backoff on 503. Key never in browser.

fix_validation.py

Sandbox engine. ast.parse() syntax check. SAFE_BUILTINS restricted exec (26 functions). difflib unified diff. is_complete_cell_fix() rejects partial patches.

dataset_intelligence.py (67KB)

AI tutor engine. Explains errors in context of dataset — relates to actual column names, shapes, dtypes. Bridges dataset context with error analysis.

ml_mistake_detector.py (62KB)

Static AST analyzer — never executes code. Detects data leakage, missing train-test split, wrong metric. 1992 lines, 3 expertise levels.

cloud_policy.py

Single source of truth for cloud access. 3 modes: disabled/local_only/enabled. One function: `gemini_fallback_enabled()`. Policy exposed via /api/health.

credentials.py

OS credential store. macOS: ~/Library/Application Support/ModelMind/ chmod 600. Windows: %APPDATA%/ModelMind/ + keyring. Key never returned to browser.

ML PIPELINE & DATA LAYER

model_recommender.py (50KB)

Profiles dataset (rows, cols, missing%, feature types). Infers task type. Ranks 10+ ML models by suitability. Generates complete ready-to-run Python code per model.

preprocessing_advisor.py (39KB)

Smart preprocessing engine. Per-column strategy (ID detection, missing strategy, encoding advice). Supports basic/medium/advanced levels.

preprocessing_apply.py

Applies transformations: mean/median imputation (numerical), most_frequent/unknown (categorical). Undo via snapshot. SUPPORTED_NUMERICAL + CATEGORICAL strategies.

preprocessing_preview.py

Non-destructive preview of preprocessing result. Shows transformed sample before committing. Same logic as apply but returns preview only — no disk write.

dataset.py (27KB)

CSV/Excel/JSON loading, column profiling, missing analysis, distribution summaries, target column analysis, classification/regression inference.

model_evaluator.py (21KB)

Cross-validation, accuracy/F1/RMSE/R2, side-by-side model comparison, background async evaluation jobs, ranked report generation.

fast_hyperparameter_tuner.py (18KB)

Grid Search + Random Search in background async jobs. AutoML code adapter. Result adapter normalizes output. Smart search strategy selection.

pipeline_builder.py + verified_model_code.py

Builds sklearn pipeline from recommendations. Verified model code templates. automl_code_adapter for AutoML I/O normalization.

5 Security Architecture

Privacy-first design — credentials, request limits, sandbox isolation, cloud boundaries

🔑 OS Keychain — credentials.py

- macOS: **~/Library/Application Support/ModelMind/credentials.json** chmod 600
- Windows: **%APPDATA%/ModelMind/** + keyring
- Linux: **~/.config/ModelMind/**
- NEVER returned to browser or stored in notebook
- Personal key overrides env variable

🛡️ Request Validation

- Error payload max: **40,000 bytes**
- API key max: **4,096 bytes**
- Exact field set enforced — extras rejected 400
- All fields individually length-bounded
- Ollama responses: **64KB max**
- Gemini responses: **32KB max**

☁️ Cloud Policy — cloud_policy.py

- 3 modes: `disabled` , `local_only` , `enabled`
- Gemini requires ALL 3 gates to pass
- Invalid env values → safe default
- `gemini_fallback_enabled()` — single check function
- Policy exposed via /api/health endpoint

🚫 What NEVER Happens

Browser never receives Gemini API key. No credentials in frontend code, notebook source, or downloadable client. Ollama never auto-installed. Model (qwen2.5-coder:7b) never auto-downloaded — requires explicit `allow_download: true`. Env variables, secrets, unrelated cell outputs never sent to any AI provider.

🔐 Sandbox Isolation — fix_validation.py

Fix sandbox restricts Python to 26 safe built-ins only:

```
SAFE_BUILTINS = {
  "abs", "all", "any", "bool",
  "dict", "enumerate", "float",
  "int", "len", "list", "max",
  "min", "print", "range",
  "round", "set", "sorted",
  "str", "sum", "tuple", "zip"...
}
# No filesystem, no network, no OS calls
```

📡 Ollama Hardcoded Limits — ollama.py

```
# All limits hardcoded in ollama.py
OLLAMA_BASE_URL  = "http://127.0.0.1:11434"
STATUS_TIMEOUT   = 2.0    # seconds
STATUS_MAX_BODY  = 65_536  # 64 KB
EXPLAIN_TIMEOUT  = 60.0   # seconds
EXPLAIN_MAX_RESP = 16_384  # 16 KB
FIX_TIMEOUT      = 120.0  # seconds
FIX_MAX_BODY     = 32_768  # 32 KB
```

6 FastAPI Web Companion Backend

modelmind-source/backend/main.py — bridge between Next.js dashboard and local Jupyter

📌 Purpose

Separate **FastAPI v0.3.0** server running alongside JupyterLab at `127.0.0.1:8000`. Allows the hosted Next.js web dashboard to interact with the user's local machine — opening notebooks, uploading datasets, running ML pipeline — through a secure local API.

🛠 FastAPI Tech Stack

- **FastAPI** — REST API with OpenAPI auto-docs
- **Pydantic v2** — Request/response validation
- **CORSMiddleware** — Origin whitelist via MODELMIND_WEB_ORIGINS
- **Uvicorn** — ASGI server
- **jupyter-server** — Lists running Jupyter sessions
- **jupyter-core** — Data directory resolution

🌐 CORS & Origin Security

- Origins configured via `MODELMIND_WEB_ORIGINS` env var
- Default: `localhost:3000` only (development)
- URL validated — no wildcards, no trailing slashes
- Binds only to `127.0.0.1` — never internet-exposed
- Separate from Jupyter token auth system

Method	FastAPI Endpoint	What It Does	Module
GET	/api/health	Companion health + Jupyter session status	main.py
GET	/api/jupyter/open	Redirect to Jupyter with auth token	main.py
POST	/api/dataset/upload	Accept CSV/Excel file, save to workspace	runtime/files.py
GET	/api/dataset/list	List all datasets in workspace	runtime/files.py
POST	/api/dataset/analyze	Full dataset profiling (shape, types, missing)	intelligence/dataset.py
POST	/api/preprocessing/analyze	Smart preprocessing recommendations	intelligence/preprocessing_advisor.py
POST	/api/preprocessing/apply	Apply transformations and export cleaned CSV	intelligence/preprocessing_apply.py
POST	/api/debug/analyze	Error analysis for web-submitted code	debugger/engine.py
POST	/api/debug/ml-mistakes	AST-based ML methodology mistake check	debugger/ml_mistake_detector.py
POST	/api/ai/gemini	Cloud AI with policy gate check	ai/gemini_fallback.py
GET	/api/packages	Installed Python package list	runtime/packages.py
POST	/api/packages/install	Install Python package into local venv	runtime/packages.py

7 ML Intelligence Pipeline

End-to-end ML workflow automation — all running locally, all consent-based

📊 Dataset Analysis

- Load CSV / Excel / JSON
- Shape, dtypes, missing% per column
- Statistical summary (mean, std, quartiles)
- Correlation detection
- Classification vs regression auto-inference
- Feature-by-feature deep analysis

🧹 Preprocessing Pipeline

- **Advisor:** Per-column strategy (basic/medium/advanced)
- **Preview:** Show result before committing
- **Apply:** Mean/median imputation; most_frequent/unknown
- **Undo:** Snapshot-based rollback
- ID column auto-detection and removal

🤖 Model Recommendation

- Profiles: rows, cols, missing%, feature types
- Infers task: classification or regression
- Ranks 10+ algorithms by suitability score
- Generates complete ready-to-run Python code
- Includes train/test split, fit, evaluation

📈 Model Evaluation

- Cross-validation (configurable folds)
- Classification: accuracy, F1, precision, recall, AUC
- Regression: RMSE, MAE, R^2 score
- Side-by-side model comparison
- Background async evaluation jobs

⚙️ Hyperparameter Tuning

- Grid Search + Random Search
- Background async job execution
- Result polling via background_tuning_routes
- AutoML code adapter generates tuning code
- Smart search strategy selection

🔍 ML Mistake Detector

- Static AST — **never executes code**
- Detects: data leakage, missing train-test split
- 3 expertise levels: basic/medium/advanced
- Each finding: explanation + fix suggestion
- 1992-line comprehensive rule set

🔄 Complete ML Workflow — All Local, All Consent-Based

Upload CSV → Dataset Analysis → Preprocessing Advisor → Preview & Apply → Model Recommendation → Model Evaluation → Hyperparameter Tuning → Error Assistant → ML Mistake Detector

Every step local. Every AI call requires explicit user intent. No data leaves without consent.

8 Complete Technology Stack

Every technology with version and rationale

CORE BACKEND FRAMEWORK

Technology	Version	Role
Python	3.14.2	Primary backend language
JupyterLab	4.6.1 (pinned)	Notebook engine
Jupyter Server	2.20.0 (pinned)	Extension host + auth
Tornado	≥6.x	Async HTTP for extension routes
FastAPI	latest	Web companion REST API
Pydantic	v2	Request/response validation
Uvicorn	latest	ASGI server for FastAPI

SECURITY & SYSTEM

Technology	Role
keyring	Windows Credential Manager integration
json + chmod 600	Cross-platform credential file
ast (stdlib)	AST parsing for sandbox + mistake detector
difflib (stdlib)	Unified diff for fix preview
subprocess	Ollama CLI (with user approval only)
importlib.resources	Load model-manifest.json from package
functools.lru_cache	Cache model manifest after first load

AI & ML LIBRARIES

Technology	Role
Ollama v0.35.0	Local LLM runtime at 127.0.0.1:11434
qwen2.5-coder:7b	Primary local coding model (4.7GB, Apache-2.0)
Gemini API	Cloud fallback (gemini-3.5-flash-lite primary)
Pandas	Dataset loading, profiling, transformations
NumPy	Numerical ops, shape analysis
scikit-learn	ML model training, evaluation, pipelines
certifi	TLS certificate bundle for Gemini HTTPS

FRONTEND & BUILD

Technology	Version	Role
TypeScript	5.x	JupyterLab extension frontend
Node.js	24.11.1	Build toolchain
npm	11.6.2	Package manager
Next.js	latest	Web dashboard framework
Supabase	latest	Web dashboard auth + database

```
# Verified Developer Machine
OS:  macOS · Apple M4 · 16 GB Unified Memory
GPU: Metal backend for Ollama acceleration
RAM: 16 GB minimum for qwen2.5-coder:7b (4.7 GB)
```

9 Hackathon Judge Q&A Preparation

Top backend questions judges will ask — with strong answers ready to deliver

❓ "Why use Jupyter Server Extension instead of a standalone server?"

Jupyter Server provides built-in authentication via tokens. Every route uses `@tornado.web.authenticated` — no endpoint is reachable without a valid Jupyter session token. A standalone server would require building auth from scratch. We get enterprise-grade security for free by extending Jupyter.

❓ "How do you ensure the AI does not give a wrong fix?"

Every fix — from Ollama or Gemini — is validated in a sandbox using Python's `ast.parse()` and a restricted `exec()` with only 26 safe built-in functions. Fail → rejected. Pass → user sees a diff and must explicitly click "Apply Fix". The original cell is preserved until user acts. No auto-application ever.

❓ "How is this different from jupyter-ai?"

jupyter-ai is a generic chat — you copy-paste errors manually. ModelMind automatically captures the failing cell, runs ML-specific rule-based diagnostics first (90% of common ML errors solved without AI), validates fixes in sandbox, and stores credentials in OS keychain. jupyter-ai has none of these: no auto-capture, no deterministic-first pipeline, no sandbox validation, no OS keychain.

❓ "What if the user has no internet?"

ModelMind works 100% offline. Local Ollama + qwen2.5-coder:7b runs entirely on the user's machine. Deterministic diagnostics need no network. Gemini contacts cloud only if all 3 gates pass — one requires user opt-in. Default is fully local-first.

❓ "What model do you use and why?"

qwen2.5-coder:7b — 7B parameter coding-specialized model by Alibaba Qwen team, Apache-2.0 licensed, ~4.7 GB on disk, 32K context window. Outperforms generic LLMs on coding tasks, runs well on Apple M4/16 GB RAM, Apache-2.0 allows commercial use, manifest is version-locked (no floating dependencies).

✅ One-Line Backend Summary (memorize this)

"The backend is a two-layer Python system — a Jupyter Server extension with 20+ authenticated Tornado routes and a FastAPI companion server — implementing a 3-tier AI pipeline: deterministic rules → local Ollama → Gemini with 3 consent gates, with sandboxed fix validation and OS keychain credential security throughout."

🏆 Strongest Differentiator to Highlight

"We sandbox-validate every AI fix before showing it to the user. No other notebook — Colab, Kaggle, Databricks, Deepnote — does this. Our ML Mistake Detector uses static AST analysis to find methodology errors like data leakage without executing any code. That is a real safety feature."